# 大语言模型微调

罗从良 (rontomai@gmail.com)

2023 年 12 月 3 日

本文针对大语言模型 llama2 微调进行操作讲述。个人水平有限，在讲述仅做个基本介绍，还望理解。

[25]:
```
!which python
```

```
huggingface/tokenizers: The current process just got forked, after parallelism
has already been used. Disabling parallelism to avoid deadlocks…
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true |
false)
/data/lcl/anaconda3/envs/chinese_llama2/bin/python
```

[ ]:
```
import os
import bitsandbytes as bnb
from functools import partial

import torch
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training,␣
 ↪AutoPeftModelForSeq2SeqLM
from transformers import AutoModelForCausalLM, AutoTokenizer, set_seed,␣
 ↪Trainer, TrainingArguments, BitsAndBytesConfig, \
    DataCollatorForLanguageModeling, Trainer, TrainingArguments
```

[27]:
```
def load_model(model_name, bnb_config):
    n_gpus = torch.cuda.device_count()
    max_memory = f'{40960}MB'

    # 正常情况下，我们直接加载 model_name,quantization_config 进行优化加载配置
    model = AutoModelForCausalLM.from_pretrained(
```

```
        model_name,
        quantization_config=bnb_config,
        device_map='auto',# 高效地将模型分配给可用资源。
        max_memory={i: max_memory for i in range(n_gpus)},
    )
    tokenizer = AutoTokenizer.from_pretrained(model_name, use_auth_token=True)

    # Needed for LLaMA tokenizer
    tokenizer.pad_token = tokenizer.eos_token

    return model, tokenizer
```

[28]:
```
from datasets import load_dataset

dataset = load_dataset("/data/lcl/LLM/llama/databricks-dolly-15k",␣
 ↪split="train")
```

[29]:
```
# 查看数据集
print(f'Number of prompts: {len(dataset)}')
print(dataset, type(dataset))
```

```
Number of prompts: 15011
Dataset({
    features: ['instruction', 'context', 'response', 'category'],
    num_rows: 15011
}) <class 'datasets.arrow_dataset.Dataset'>
```

[30]:
```
# 探索数据集里具体的内容
for e in dataset:
    print(e['instruction'])
```

```
When did Virgin Australia start operating?
Which is a species of fish? Tope or Rope
Why can camels survive for long without water?
Alice's parents have three daughters: Amy, Jessy, and what' s the name of the
third daughter?
When was Tomoaki Komorida born?
If I have more pieces at the time of stalemate, have I won?
```

```
Given a reference text about Lollapalooza, where does it take place, who started
it and what is it?
Who gave the UN the land in NY to build their HQ
Why mobile is bad for human
Who was John Moses Browning?
Who is Thomas Jefferson?
....
```

[31]:
```
# 预处理数据集

# instrcuction fine-tuning(指令微调) 是一种常用的技术，用于针对特定的下游用例微调基
础 LLM。

# 按如下方式设置提示的格式:
```

```
Below is an instruction that describes a task. Write a response that appropriately completes th

### Instruction:
Sea or Mountain

### Response:
I believe Mountain are more attractive but Ocean has it's own beauty and this tropical weather

### End
```

[32]:
```
# 通过主题标签分隔每个提示部分
def create_prompt_formats(sample):
    """
        格式化为 ('instruction', 'context', 'response')
        然后组成为新的特征行。
    """
    INTRO_BLURB = "Below is an instruction that describes a task. Write a␣
 ↪response that appropriately completes the request."
    INSTRUCTION_KEY = "### Instruction:"
    INPUT_KEY = "Input:"
    RESPONSE_KEY = "### Response:"
    END_KEY = "### End"
```

```
    blurb = f"{INTRO_BLURB}"
    instruction = f"{INSTRUCTION_KEY}\n{sample['instruction']}"
    input_context = f"{INPUT_KEY}\n{sample['context']}" if sample['context']␣
 ↪else None
    response = f"{RESPONSE_KEY}\n{sample['response']}"
    end = f"{END_KEY}"

    parts = [part for part in [blurb, instruction, input_context, response,␣
 ↪end] if part]

    formatted_prompt = "\n\n".join(parts)

    sample["text"] = formatted_prompt

    return sample
```

[33]:
```
# 使用 model tokenizer 将这些 prompts into tokenized ones
# 目标是创建长度均匀的输入序列（适用于微调语言模型，因为它可以最大限度地提高效率并最
小化计算开销），并且不得超过模型的最大标记限制。

def get_max_length(model):
    conf = model.config
    max_length = None
    for length_setting in ["n_positions", "max_position_embeddings",␣
 ↪"seq_length"]:
        max_length = getattr(model.config, length_setting, None)
        if max_length:
            print(f"找到最大长度: {max_length}")
            break
    if not max_length:
        max_length = 1024
        print(f"使用默认最大长度: {max_length}")
    return max_length

# 准备 batch
```

```
# 对一个批次的数据进行预处理
def preprocess_batch(batch, tokenizer, max_length):
    """
        Tokenizer a batch
    """
    # batch["text"] 表示批次中的文本数据，max_length 表示标记化后每个文本序列的最大长度，trucation=True 表示文本长度超过了 max_length 则进行截取
    # 返回标记后的文本
    return tokenizer(
        batch["text"],
        max_length = max_length,
        truncation=True,
    )

# 准备数据集
def preprocess_dataset(tokenizer: AutoTokenizer, max_length: int, seed, dataset:
 ↪ str):
    """
        准备数据集
    """
    print("准备数据集")
    dataset = dataset.map(create_prompt_formats)#,batched=True)

    # 对数据集中的每个 batch 进行预处理，移除'instruction', 'context',␣
 ↪'response', 'category'
    # from functools import partial
    """
        def add(x, y):
            return x + y

        add_5 = partial(add, 5)
        print(add_5(3)) # 输出 8
        print(add_5(7)) # 输出 12
    """
    _preprocessing_function = partial(preprocess_batch, max_length=max_length,␣
 ↪tokenizer=tokenizer)
```

```
    dataset = dataset.map(
        _preprocessing_function,
        batched=True,
        remove_columns=["instruction", "context", "response", "text",␣
↪"category"],
    )

    # Filter out samples that have input_ids exceeding max_length
    dataset = dataset.filter(lambda sample: len(sample["input_ids"]) <␣
↪max_length)

    # Shuffle dataset
    dataset = dataset.shuffle(seed=seed)

    return dataset
```

创建 bitsandbytes 配置，这将允许我们以 4 位加载我们的 LLM。这样，我们可以将使用的内存除以 4，并在较小的设备上导入模型。为了节省内存，我们选择应用 bfloat16 计算数据类型和嵌套量化。

[34]:
```
def create_bnb_config():
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type='nf4',
        bnb_4bit_compute_dtype=torch.bfloat16,
    )
    return bnb_config
```

[35]:
```
# 为了使用 LoRa 方法，我们需要将模型包装为 PeftModel
# 因此，我们需要实现 LoRa 配置
# 函数需要目标模块来更新必要的矩阵。

def create_peft_config(modules):
    """
        为您的模型创建参数高效的微调配置
    :param modules: 要应用 Lora 的模块的名称
```

```
    """
    config = LoraConfig(
        r=16, # 更新矩阵维度 16
        lora_alpha=64, # 表示缩放参数为 64
        target_modules=modules, # 表示要应用 Lora 的模块名称
        lora_dropout=0.1, #  layers 的 dropout 概率为 0.1
        bias="none", # 表示偏差为 none
        task_type="CAUSAL_LM",
    )
    return config
```

[36]:
```
# 函数用来获取 lora 操作系列模块
def find_all_linear_names(model):
    cls = bnb.nn.Linear4bit #if args.bits == 4 else (bnb.nn.Linear8bitLt if␣
 ↪args.bits == 8 else torch.nn.Linear)
    lora_module_names = set()
    for name, module in model.named_modules():
        if isinstance(module, cls):
            names = name.split('.')
            lora_module_names.add(names[0] if len(names) == 1 else names[-1])


    if 'lm_head' in lora_module_names:  # needed for 16-bit
        lora_module_names.remove('lm_head')
    return list(lora_module_names)
```

[37]:
```
# 创建一个辅助函数用来查看模型中的可训练参数
def print_trainable_parameters(model,use_4bit=False):
    """
        打印模型中的训练参数
    """
    trainable_params = 0
    all_param = 0
    for _, param in model.named_parameters():
        num_params = param.numel()
        # 如果使用 DS Zero 3 并且权重初始化为空。
        if num_params == 0 and hasattr(param, "ds_numel"):
            num_params = param.ds_numel
```

```
        all_param += num_params
        if param.requires_grad:
            trainable_params += num_params
    if use_4bit:
        trainable_params /= 2
    # print(f"all params: {all_param:, d} || trainable params:␣
↪{trainable_params:, d} || trainable%: {100 * trainable_params / all_param}")
```

[38]:
```
# 加载预训练模型和配置
model_name = "meta-llama/Llama-2-7b-hf"

bnb_config = create_bnb_config()

model, tokenizer = load_model(model_name, bnb_config)
```

```
Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]
```

[39]:
```
# 准备数据集
max_length = get_max_length(model)
dataset = preprocess_dataset(tokenizer, max_length, 515, dataset)
```

```
找到最大长度: 4096
准备数据集
```

[40]:
```
def train(model, tokenizer, dataset, output_dir):
    # 预处理启动梯度检查点技术，以在微调过程中减少内存使用
    model.gradient_checkpointing_enable()

    # 2 - Using the prepare_model_for_kbit_training method from PEFT
    model = prepare_model_for_kbit_training(model)

    # Get lora module names
    modules = find_all_linear_names(model)

    # Create PEFT config for these modules and wrap the model to PEFT
    peft_config = create_peft_config(modules)
    model = get_peft_model(model, peft_config)
```

```
    # Print information about the percentage of trainable parameters
    print_trainable_parameters(model)

    # Training parameters
    trainer = Trainer(
        model=model,
        train_dataset=dataset,
        args=TrainingArguments(
            per_device_train_batch_size=1,
            gradient_accumulation_steps=4,
            warmup_steps=2,
            max_steps=20,
            learning_rate=2e-4,
            fp16=True,
            logging_steps=1,
            output_dir="outputs",
            optim="paged_adamw_8bit",
        ),
        data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
    )

    model.config.use_cache = False  # re-enable for inference to speed up␣
↪predictions for similar inputs

    ### SOURCE https://github.com/artidoro/qlora/blob/main/qlora.py
    # 在训练之前对数据进行检查

    dtypes = {}
    for _, p in model.named_parameters():
        dtype = p.dtype
        if dtype not in dtypes: dtypes[dtype] = 0
        dtypes[dtype] += p.numel()
    total = 0
    for k, v in dtypes.items(): total+= v
    for k, v in dtypes.items():
```

```
        print(k, v, v/total)

    do_train = True

    # 进行训练
    print("训练...")

    if do_train:
        train_result = trainer.train()
        metrics = train_result.metrics
        trainer.log_metrics("train", metrics)
        trainer.save_metrics("train", metrics)
        trainer.save_state()
        print(metrics)

    ###

    # Saving model
    print("Saving last checkpoint of the model...")
    os.makedirs(output_dir, exist_ok=True)
    # 为了稍后加载和使用模型进行推理，我们使用了该 trainer.model.
↪save_pretrained(output_dir) 函数，该函数保存了微调模型的权重、配置和分词器文件。
    trainer.model.save_pretrained(output_dir)

    # Free memory for merging weights
    del model
    del trainer
    torch.cuda.empty_cache()
```

[41]:
```
output_dir = "results/llama2/final_checkpoint"
train(model, tokenizer, dataset, output_dir)
```

```
Detected kernel version 4.15.0, which is below the recommended minimum of 5.5.0;
this can cause the process to hang. It is recommended to upgrade the kernel to
the minimum version or higher.
You're using a LlamaTokenizerFast tokenizer. Please note that with a fast
tokenizer, using the `__call__` method is faster than using a method to encode
the text followed by a call to the `pad` method to get a padded encoding.

torch.float32 302387200 0.08541070604255438
torch.uint8 3238002688 0.9145892939574456
Training…

<IPython.core.display.HTML object>

***** train metrics *****
  epoch                    =       0.01
  total_flos               =   333945GF
  train_loss               =     1.3274
  train_runtime            = 0:01:07.10
  train_samples_per_second =      1.192
  train_steps_per_second   =      0.298
{'train_runtime': 67.1027, 'train_samples_per_second': 1.192,
'train_steps_per_second': 0.298, 'total_flos': 358571599552512.0, 'train_loss':
1.327436164021492, 'epoch': 0.01}
Saving last checkpoint of the model…
```

[45]:
```
# 一旦我们有了微调的权重，我们就可以构建微调的模型，并将其保存到一个新目录，以及其关
联的分词器。通过执行这些步骤，我们可以为推理提供内存高效的微调模型和分词器!
from peft import AutoPeftModelForCausalLM
model = AutoPeftModelForCausalLM.from_pretrained(output_dir, device_map="auto",␣
 ↪torch_dtype=torch.bfloat16)
model = model.merge_and_unload()

output_merged_dir = "results/llama2/final_merged_checkpoint"
os.makedirs(output_merged_dir, exist_ok=True)
model.save_pretrained(output_merged_dir, safe_serialization=True)
```

```
# save tokenizer for easy inference
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.save_pretrained(output_merged_dir)
```

```
Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]
```

[45]:
```
('results/llama2/final_merged_checkpoint/tokenizer_config.json',
 'results/llama2/final_merged_checkpoint/special_tokens_map.json',
 'results/llama2/final_merged_checkpoint/tokenizer.model',
 'results/llama2/final_merged_checkpoint/added_tokens.json',
 'results/llama2/final_merged_checkpoint/tokenizer.json')
```